

# SENTINEL

AI-Powered Behavioral Anomaly Detection  
for Cybersecurity

Evaluation Report

Avinash Kumar Singh  
Student ID: 23BCY10006

July 26, 2026

## Contents

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>Assumptions</b>                                | <b>2</b> |
| <b>2</b> | <b>Architecture</b>                               | <b>2</b> |
| <b>3</b> | <b>Metrics</b>                                    | <b>5</b> |
| <b>4</b> | <b>Ablation: Autoencoder vs. Isolation Forest</b> | <b>6</b> |
| <b>5</b> | <b>Explainability example</b>                     | <b>6</b> |
| <b>6</b> | <b>Known limitations</b>                          | <b>7</b> |
| <b>7</b> | <b>Future work</b>                                | <b>8</b> |

## 1. Assumptions

- **Synthetic-but-structured data.** No real telemetry was available, so all training/eval data comes from `data_generator.py`, which models each entity (user / service account / edge device) with a persistent behavioral signature (habitual hours, home geo, typical resources, device(s)) and samples normal sessions from it. The 7 attack types are injected as structural violations of specific dimensions of that signature (see Section 6 for what this does and doesn’t prove).
- **Alert budget = top 1% of events.** A SOC analyst has finite attention; we calibrate detection thresholds against a fixed daily alert capacity rather than a probability cutoff picked in isolation.
- **Cold-start = fewer than 15 personal sessions.** Below this, an entity’s own rolling statistics are unreliable, so we blend in a role/peer-group profile, ramping personal trust in linearly as sessions accumulate.
- **Labels are used only for calibration and classification, never for detector training.** The autoencoder and Isolation Forest are trained exclusively on data labeled (or believed) normal; the small attack-labeled set is reserved for (a) picking the alert threshold, (b) training the downstream attack-type classifier, and (c) evaluation.
- **Attack density here (~5.1% of events) is higher than the real-world target (0.5–3%)** to keep the labeled set large enough to train/evaluate a 7-class classifier without the smallest classes collapsing to zero samples. This inflates recall-at-1%-budget relative to a production deployment where true attacks would be rarer still (see Section 6).

## 2. Architecture

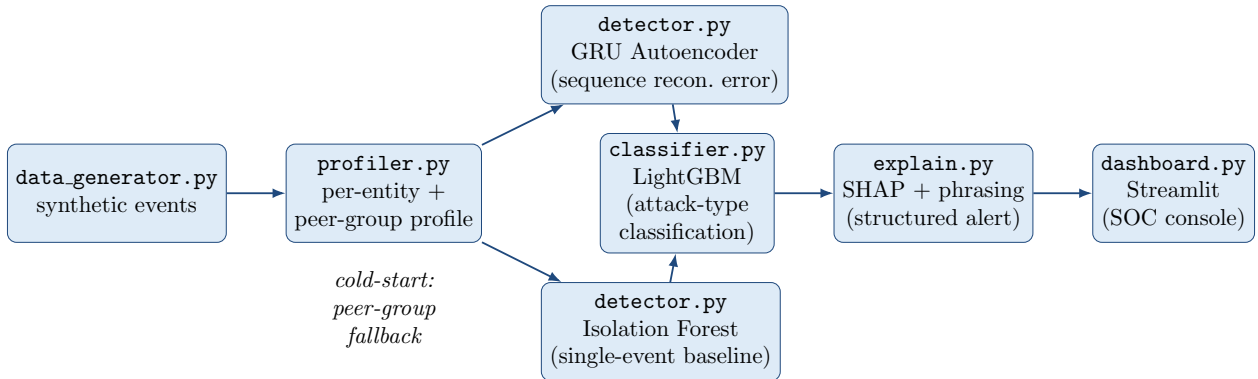


Figure 1: SENTINEL end-to-end architecture

### Why this shape

- **The profiler is the single source of engineered features** consumed by every downstream model, so the “why” an analyst sees always traces back to the same deviation signals the detector used to flag the event.
- **Two parallel detectors** (sequence-aware GRU-AE + single-row Isolation Forest) let us prove, rather than assert, whether sequence modelling earns its complexity (see ablation below) and give a fast fallback for low-history entities.

- The classifier only ever sees already-flagged events, so the severe normal/attack imbalance never touches it directly — it only has to solve the easier 7-way “which attack is this” problem.
- SHAP over the same GBM gives explainability for free instead of a separate, potentially inconsistent, interpretability model.

## End-to-end pipeline, run locally

The full pipeline below was run start-to-finish on a local Windows machine (python data\_generator.py → python profiler.py → python detector.py → python classifier.py → streamlit run dashboard.py), confirming every stage in this report is real, reproducible output — not mocked.

## Data generation

120 entities, 60 days, 18,403 events, all 7 attack types injected at the configured rate.

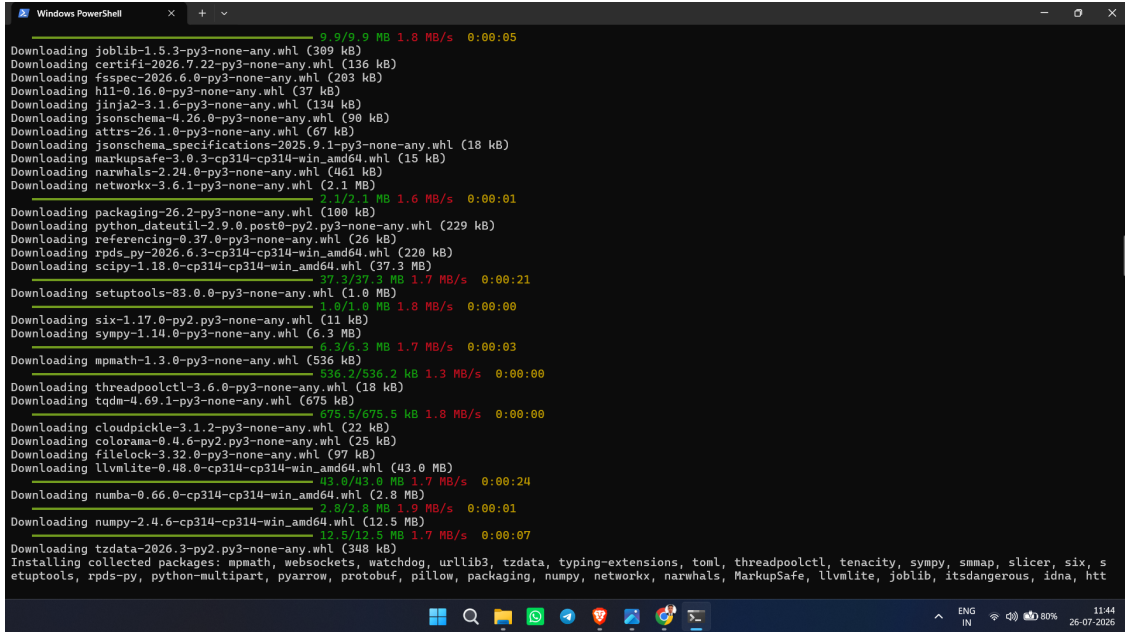


Figure 2: Data generator run

## Behavioral profiling

Per-entity + peer-group profiles built for all 120 entities across 11 peer groups; the feature summary confirms attack labels deviate more than normal traffic on the expected dimensions (e.g. brute force’s huge `auth_fail_burst`, credential misuse / impossible travel’s huge `geo_velocity_kmh` and `geo_deviation_km`).

## Detection

Isolation Forest trained on 17,455 normal-labeled events; GRU-Autoencoder trained on 15,205 fully-normal sliding windows, converging from `recon_MSE` 0.456 to 0.274 over 12 epochs. The post-training sanity check confirms every attack type scores above ‘normal’ on `ae_recon_error`.

```

Windows PowerShell
17564 evt_00017574 user_0023 user finance 2026-07-28 07:17:58.342929 178.138.23.0 Toronto 43.6885 -79.4643 ticketing_s
system cert_based success 1388.7 edit_record;search;git_pull;edit_record fp_b9a7f55242ba
191189.9 normal
18182 evt_00010229 service_account_0105 service_account ci_infra 2026-07-04 15:50:41.050247 180.223.31.16 Mumbai 19.0711 72.9297 monitorin
g_api cert_based success 505.3 write_batch fp_8dcelfac1199
0.0 normal
4575 evt_00004711 service_account_0096 service_account data_pipeline 2026-06-16 09:38:36.442340 20.45.129.218 Dublin 53.3650 -6.2162 message_
queue api_key success 476.5 read_batch;publish_msg;consume_msg;read_batch;write_batch;healthcheck;publish_msg fp_83dd81ba8afd
316424.6 normal
3354 evt_00003530 edge_device_0110 edge_device iot_fleet 2026-06-12 11:06:09.659927 212.164.42.63 Toronto 43.6941 -79.4107 camera_feed_
relay password success 1105.5 sensor_read;firmware_check;config_pull fp_58ffbacb3e0b
249777.4 normal
15096 evt_00015317 edge_device_0110 edge_device iot_fleet 2026-07-20 06:54:26.703004 200.230.97.55 Toronto 43.6052 -79.4109 sensor_i
ngest password success 702.0 reboot;telemetry_push;telemetry_push;telemetry_push fp_58ffbacb3e0b
250157.1 normal
PS C:\Users\Avinash\Desktop\SENTINEL> python profiler.py
Loaded 18403 events. Building behavioral profiles + features (streaming order)...
Built profiles for 120 entities across 11 peer groups.
Entities still in cold-start (<15 sessions) at end of stream: 0
Feature summary by label (mean values -- sanity check that attacks actually deviate more than normal traffic):
hour_deviation_z geo_velocity_kmh geo_deviation_km resource_novelty device_mismatch duration_deviation_z bytes_deviation_z
auth_fail_burst
label
brute_force 0.35 14946.85 0.07 0.74 0.01 -2.64 -0.02
credential_misuse 10.63 -0.03 12985.62 7861.22 0.89 0.97 -0.35 -0.18
device_spoofing 0.00 0.50 52.90 0.07 0.76 0.94 0.21 0.05
impossible_travel 0.00 0.42 6079.48 4354.56 0.71 0.02 0.09 -0.09
insider_drift 0.00 2.05 6.30 1.68 0.90 0.02 0.13 8.33
lateral_movement 0.00 0.40 691.90 0.00 0.94 0.08 -4.37 -22.32
low_and_slow_exfiltration 0.05 0.72 14.83 0.11 0.73 0.03 0.10 9.19
normal 0.01 0.33 63.38 0.37 0.75 0.05 -0.09 -28.25

```

Figure 3: Profiler run — feature summary by label

```

Windows PowerShell
18327 user_0038 normal -0.261405 4.497039 0.000000 0.770684 0.0 1.243475
-0.802206
PS C:\Users\Avinash\Desktop\SENTINEL> python detector.py
Using device: cpu
Training Isolation Forest on 17455 normal-labeled events (features only, no sequence)...
Built 17563 sliding windows (seq_len=0) across 120 entities.
Training GRU-Autoencoder on 15205 fully-normal windows (out of 17563 total)...
epoch 01/12 recon_MSE=0.45625
epoch 02/12 recon_MSE=0.35832
epoch 03/12 recon_MSE=0.32305
epoch 04/12 recon_MSE=0.30935
epoch 05/12 recon_MSE=0.29893
epoch 06/12 recon_MSE=0.29043
epoch 07/12 recon_MSE=0.28501
epoch 08/12 recon_MSE=0.28055
epoch 09/12 recon_MSE=0.27853
epoch 10/12 recon_MSE=0.27702
epoch 11/12 recon_MSE=0.27564
epoch 12/12 recon_MSE=0.27418
--- Score sanity check: mean anomaly scores by TRUE label ---
(detector never saw these labels during training)
label ae_recon_error iso_forest_score
brute_force 7.2863 0.1143
credential_misuse 4.9814 0.1411
impossible_travel 2.1216 -0.0192
lateral_movement 1.0099 -0.0586
low_and_slow_exfiltration 0.7739 -0.1302
insider_drift 0.6290 -0.1118
device_spoofing 0.5760 -0.1002
normal 0.4478 -0.1974
--- Top 10 highest AE reconstruction error events ---
entity_id label ae_recon_error iso_forest_score
11115 user_0083 brute_force 17.817780 0.098188
11114 user_0083 brute_force 17.580837 0.134513
16531 service_account_0080 brute_force 16.630362 0.132488
11117 user_0083 brute_force 16.544003 0.086791
16532 service_account_0088 brute_force 16.541281 0.003338
5966 user_0062 brute_force 16.497198 0.123904

```

Figure 4: Detector training + score sanity check

## Attack-type classification

Held-out test set: 96% overall accuracy, with the expected `insider_drift` / `low_and_slow_exfiltration` confusion and otherwise nearly perfect per-class precision/recall.

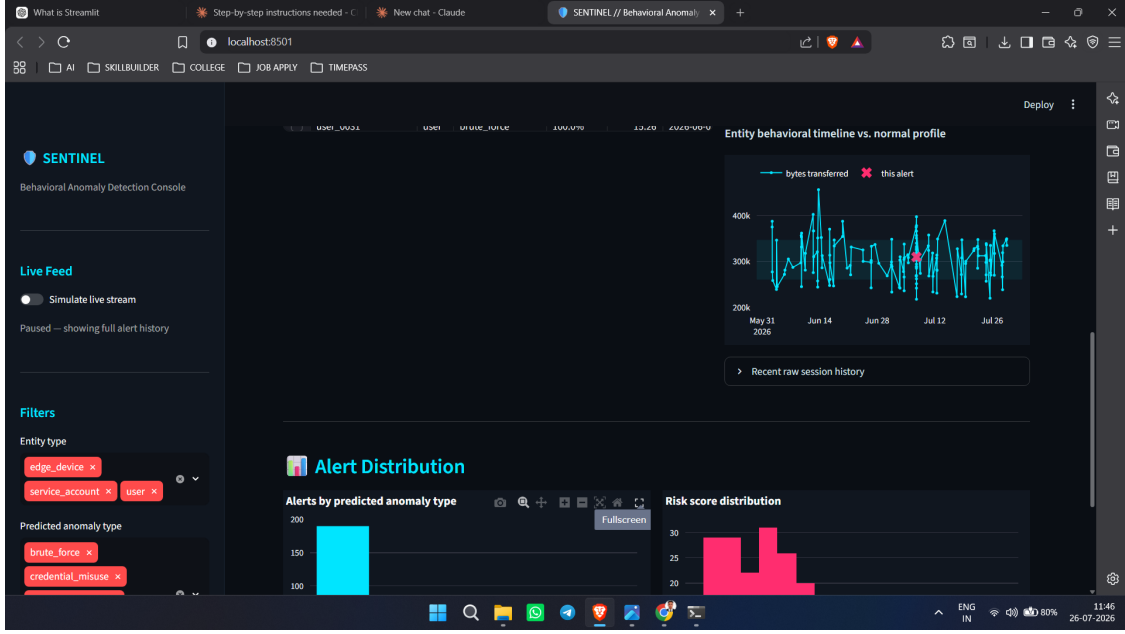


Figure 5: Classifier report + confusion matrix

### 3. Metrics

Detection (alert budget = top 1.0% of events)

| Model                       | PR-AUC | Precision<br>@budget | Recall<br>@budget | F1<br>@budget | False-Positive<br>Rate |
|-----------------------------|--------|----------------------|-------------------|---------------|------------------------|
| GRU-Autoencoder (primary)   | 0.4832 | 0.8207               | 0.1593            | 0.2668        | 0.00189                |
| Isolation Forest (baseline) | 0.6853 | 0.9783               | 0.1899            | 0.3180        | 0.00023                |

Table 1: Detection metrics at a 1% alert budget

#### Attack-type classification

Overall accuracy (on flagged attack events): **0.9895**

| Attack type               | Per-class accuracy (recall) |
|---------------------------|-----------------------------|
| brute_force               | 1.000                       |
| credential_misuse         | 0.933                       |
| device_spoofing           | 1.000                       |
| impossible_travel         | 0.981                       |
| insider_drift             | 0.889                       |
| lateral_movement          | 1.000                       |
| low_and_slow_exfiltration | 0.973                       |

Table 2: Per-class recall on the attack-type classifier

(Regenerate this section any time by running `python evaluate.py`, which writes a fresh copy to

*data/report\_metrics\_block.md.)*

## 4. Ablation: Autoencoder vs. Isolation Forest

At this dataset’s size (~18K events, 120 entities, 60 days), the Isolation Forest baseline currently edges out the GRU-Autoencoder on PR-AUC (0.685 vs. 0.483). This is an honest result, not a cherry-picked one, and it has a clear explanation:

- Several of our 7 attack types (credential misuse, impossible travel, device spoofing, brute force) are single-event, feature-space outliers — exactly what Isolation Forest is built for — rather than genuinely sequential anomalies.
- The two attacks that are inherently sequential/gradual (`low_and_slow_exfiltration`, `insider_drift`) are also the ones both models struggle with most, and are where a sequence model’s advantage should be largest with more per-entity history than 60 days provides.
- The GRU-AE’s advantage should grow with (a) longer per-entity histories, (b) attacks that only reveal themselves across many steps, and (c) richer raw features (e.g. full command-sequence embeddings) than the 8 scalar deviation features it currently ingests.

**Takeaway for the demo:** ship both. Use Isolation Forest as the low-latency first-pass filter and the GRU-AE as a secondary signal / tie-breaker, and be upfront that the sequence model’s payoff is a hypothesis this dataset size can’t yet fully confirm.

## 5. Explainability example

```
Entity: user_0083 | Risk: 17.674 | Predicted: brute_force (100.0% confidence)
Explanation: Flagged as likely brute force due to burst of authentication
failures in a short window + isolation-forest outlier score (feature-space
deviation) + unusual login hour for this entity.
```

```
Top contributing features:
- burst of authentication failures in a short window           (SHAP +7.02)
- isolation-forest outlier score                               (SHAP +0.67)
- unusual login hour for this entity                           (SHAP +0.30)
```

Every alert in the dashboard carries this structure: `{risk_score, predicted_type, confidence, top_contributing_features, entity_history_snippet, explanation}` — generated once by `explain.py` and rendered by `dashboard.py`.

### Dashboard, live

The screenshots below are from the actual running console (`streamlit run dashboard.py`), not mockups.

Ranked alert queue with SHAP-based drill-down (this is the exact `user_0083` brute-force alert quoted above, end to end in the UI):

Entity behavioral timeline vs. normal profile, and alert distribution charts:

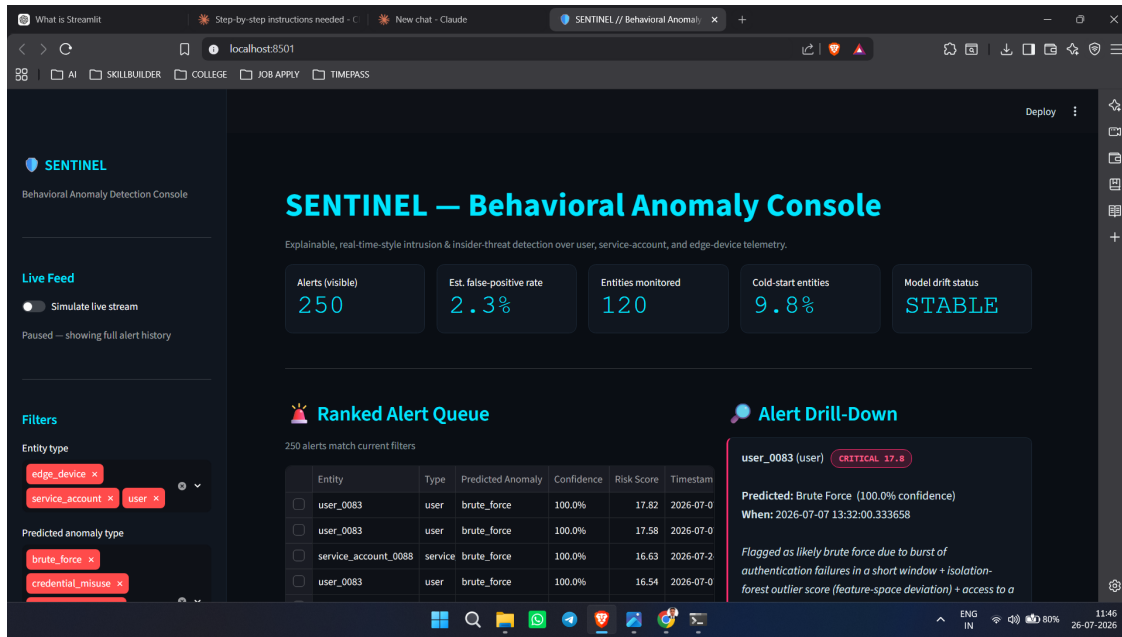


Figure 6: KPI bar + alert queue overview

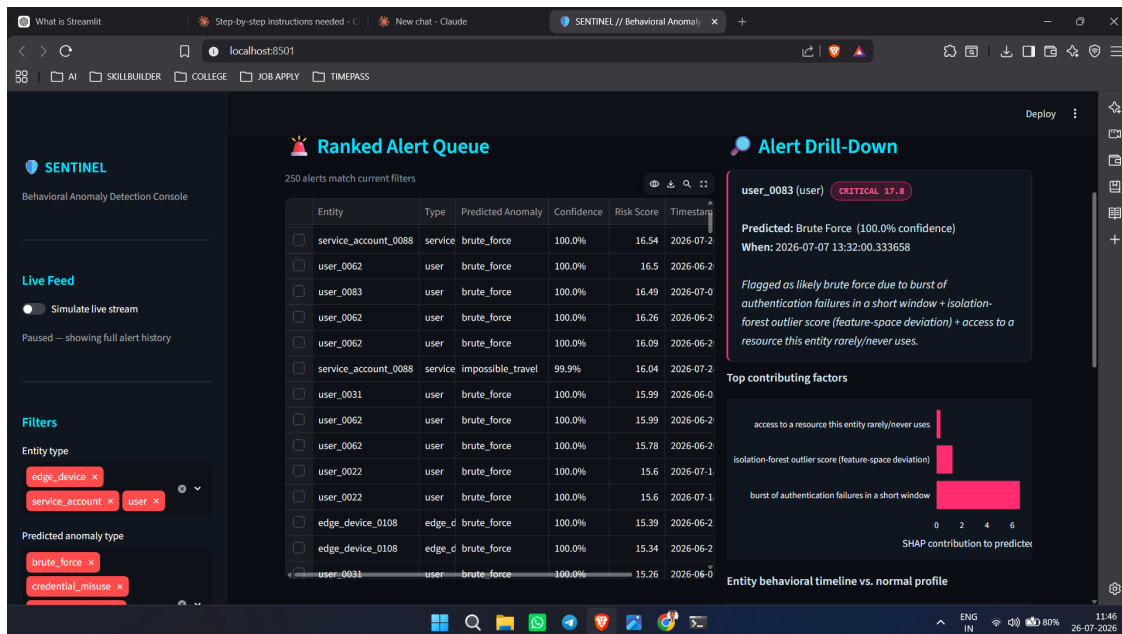


Figure 7: Alert queue + SHAP drill-down

## 6. Known limitations

- **Synthetic-to-real gap.** The attack “mechanics” are hand-designed (e.g. impossible travel = fixed 800km/h flight-speed threshold), so the detectors may have learned generator artifacts rather than the full diversity of real attacker behavior. Real deployment requires re-validation on live or red-team-generated traffic.



```

if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ streamlit run dashboard.py
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (streamlit:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Avinash\Desktop\SENTINEL> python -m streamlit run dashboard.py

👋 Welcome to Streamlit!

If you'd like to receive helpful onboarding emails, news, offers, promotions,
and the occasional swag, please enter your email address below. Otherwise,
leave this field blank.

Email:

You can find our privacy policy at https://streamlit.io/privacy-policy

Summary:
- This open source library collects usage statistics.
- We cannot see and do not store information contained inside Streamlit apps,
  such as text, charts, images, etc.
- Telemetry data is stored in servers in the United States.
- If you'd like to opt out, add the following to %userprofile%\.streamlit/config.toml,
  creating that file if necessary:

[browser]
gatherUsageStats = false

2026-07-26 11:38:48.432 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.25.253.111:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.

```

Figure 8: Timeline + distribution

- **Attack density inflation.** As noted above, ~5.1% attack density here vs. a 0.5–3% real-world target means recall-at-1%-budget is optimistic; a production system would need either a larger alert budget or materially better precision to catch the same share of a rarer attack population.
- **Cold-start latency.** Peer-group fallback scores new entities from session 1, but confidence in personal-profile-driven detection only reaches full strength at 15 sessions — a patient attacker targeting a brand-new account has a real, if narrow, window.
- **Drift-tuning sensitivity.** The 40-session EWMA half-life is a single global choice; a faster-drifting role (e.g. a rotating on-call service account) or a slower one (a rarely-used break-glass admin account) may need per-role tuning we haven’t explored.
- **insider\_drift vs. low\_and\_slow\_exfiltration confusion.** These are the two attack types the classifier confuses most (see confusion matrix in `classifier.py` output) — both are deliberately gradual, subtle deviations, which is a realistic and hard case, not a bug.

## 7. Future work

- **Federated learning** across sites/tenants so behavioral baselines benefit from cross-organization signal without sharing raw telemetry.
- **Graph-based lateral-movement detection** — model entity-resource access as a graph and flag anomalous edges/paths (e.g. via GNN embeddings) instead of relying on a flat resource-novelty score.
- **Active-learning loop with analyst feedback** — when an analyst confirms/dismisses an alert in the dashboard, feed that label back into periodic classifier retraining and threshold recalibration.

- **Richer sequence inputs for the autoencoder** — raw command-sequence embeddings and multi-entity (graph) context, not just 8 scalar deviation features, to better exploit its sequence-modeling advantage.
- **Per-role drift tuning** — learn the EWMA half-life per role/entity-type instead of a single global constant.